F02 | Dockerfile 怎样把文件变成运行模板

本章追的是：**明确构建输入 → 在镜像环境执行构建步骤 → 得到可识别的 image → 启动时采用默认命令。** 构建发生在哪里、启动发生在哪里，会决定下一步选什么操作。

builder 从哪取文件，怎样限制输入？

当前问题	关系或短用法	记到什么程度
`docker build ... .` **最后的点是什么？**	`.` 选择当前目录为 build context，即本次本地构建输入范围；`-t` 为产物指定 tag。	认熟“构建说明”和“可见输入范围”两件事；换 Dockerfile 不会自动改 context。
`COPY` **能随意取电脑上任何文件吗？**	普通本地 COPY 的来源受 context 与排除规则限制；目标是镜像中的路径。	沿“主机输入 → image 目标”找文件，不把两端路径混成同一位置。
为什么排除虚拟环境、数据库和备份？	`.dockerignore` 在构建输入阶段排除匹配内容；配套还排除 `.env`、缓存和 tests。	理解排除能减少无关输入与误带数据；具体规则查文件。
秘密先 COPY，再删除，就安全了吗？	早期层仍可能保存它。真正的 build secret 要用专门的 secret mount，不能以普通 ARG/ENV 或后续删除替代。	**进入过构建层的内容，不因后面删掉就从历史消失。** 本例无需真实凭据。

构建会产生或复用镜像与缓存，可能需要取得基础镜像；它不会自动替换正在运行的所有容器。完整 build 参数、目录材料和 `.dockerignore` 回看「builder 能看见哪些文件」。

Dockerfile 中哪些在构建，哪些等启动？

当前问题	关系或短用法	记到什么程度
FROM、COPY 分别接上什么？	FROM 选择已有基础镜像；COPY 再加入本项目文件。Python 基础镜像不自动含本项目源码。	认熟来源方向，不把 base 当最终应用。
RUN 和 CMD 是同一时机吗？	RUN 在 build 环境中执行；CMD 给容器启动提供默认命令。把长期服务写入 RUN 会让构建等待它结束。	**构建动作与运行入口直接分清。**只把安装写成 CMD，安装结束后实例也可能结束。
WORKDIR /app 改了谁的目录？	设置镜像中后续相对目标与默认进程工作目录；没有把 Windows PowerShell 切到 /app。	认熟作用位置；不是主机的 Set-Location。
ARG、ENV 怎样连接默认值？	ARG 提供构建参数；本例再用 ENV 把版本等默认设置保留到 image 配置，运行时可覆盖适合变化的值。	分清构建输入与运行环境；完整覆盖规则按需查。
USER 后非 root，为什么仍可能写不了数据？	本例先建用户、目录并设置权限，再以 10001:10001 运行；实际挂入的存储也必须可访问。	用户与路径权限一起看；不能只复制 USER 而省略条件，也不能由非 root 推断全面安全。
CMD ["python", "app.py"] 的方括号怎样读？	exec form 把可执行程序与参数分成数组元素，不要求 shell 解释整段文字。	能看出实际入口；完整 Dockerfile 查用，不要求默写。
EXPOSE 8000 已经开好主机端口了吗？	只记录预计端口元数据；应用要实际监听，运行配置要实际发布主机入口。	**声明端口、监听端口、发布端口分开。** F03 再追请求。

回看：「一份实际 Dockerfile 怎样读」。

改了文件或版本，选 rebuild、recreate 还是 restart?

当前问题	关系或短用法	记到什么程度
只改源码，为什么先复制依赖有用?	依赖清单未变时可复用安装步骤；清单变了，后续相关结果需要重新计算。	理解 build cache 复用输入对应结果；CACHED 不证明业务正确。
`--no-cache` 与 `--pull` 能互换吗?	前者要求不复用构建缓存；后者尝试取得指定 base 引用。base 固定 digest 时，pull 不会替你挑选新版本。	认得不同目的，其余参数查用；安全更新仍需明确变更和验证。
tag、base digest、最终 image ID 分别标识谁?	tag 是可移动的可读名字；base digest 固定基础内容；最终 image ID 还对应加入源码和配置后的应用产物。	**基础身份不能冒充最终应用身份。** D1 是记录实际 ID 后的讨论代号。
source、依赖或 Dockerfile 改了?	rebuild 产生新 image，再用它建立新 container。	能按变化对象选动作；build 成功不会自动更新已有实例。
只改本次端口或环境变量?	recreate 用新运行声明建立实例；源码不变时无需只为此重建 image。	分清模板变化和实例配置变化。
只是让同一实例的进程重新开始?	restart 保留原镜像与原运行配置。	认熟重启的边界；它不会重新注入刚修改的 Compose 配置。

需要确认最终产物时，用 `docker image inspect` 查实际 ID、Cmd、User；这是身份与默认入口证据，不替代 API 验收。完整命令回看「固定身份与更新并不冲突」；缓存见「为什么先复制依赖，后复制源码」，动作选择见「改了文件以后应该做什么」。